



VNiVERSIDAD
D SALAMANCA

HTML Guide for Computer Engineering

Course: Human-Computer Interaction (IPO)

Luis Augusto Silva Zendron

Universidad de Salamanca

CONTENTS

1	HTML Fundamental Concepts	3
1.1	What is a tag?	3
1.2	Self-closing tags	3
1.3	What are attributes?	3
1.4	Nesting	4
1.5	Basic structure of an HTML document	4
1.6	Comments	4
1.7	Special characters	5
1.8	Detailed Anatomy of Tags	5
2	The HTML Document: Internal Structure	9
2.1	Complete structure of an HTML document	9
2.2	What is the <head>?	9
2.3	The lang attribute	10
2.4	Meta charset	10
2.5	Meta viewport	10
2.6	Page title	10
2.7	Favicon	11
2.8	Meta description	11
2.9	Meta tags for social media	11
2.10	Linking stylesheets	11
2.11	Scripts	11
2.12	Charset (Character Encoding)	12
3	HTML Semantic Elements	15
3.1	Why does semantics matter?	15
3.2	Example: Semantic vs. Non-semantic	15
3.3	Main semantic tags	16
3.4	Structure of a semantic HTML page	17
3.5	Difference between and 	18
3.6	Accessibility and semantics	18
4	Links and Hyperlinks	19
4.1	The <a> tag	19
4.2	URLs vs. Paths	19
4.3	Link attributes	20
4.4	Open in new tab	20
4.5	Internal links within the page	20
4.6	Download files	20
4.7	Email and phone	20

4.8	Best practices	21
4.9	Difference between paths	21
5	Tables in HTML	23
5.1	Basic table structure	23
5.2	Table tags	23
5.3	Table with complete structure	24
5.4	Cell attributes	24
5.5	Merged cells	24
5.6	Accessibility in tables	25
5.7	Complex tables	25
5.8	When to use tables?	25
6	Your First HTML Project	27
6.1	Project Objective	27
6.2	Project Structure	27
6.3	The index.html file	27
6.4	Other pages	29
6.5	Steps to create your project	30
6.6	Next steps	30
6.7	Review Checklist	30
7	Forms in HTML	31
7.1	Why are forms important?	31
7.2	Basic form structure	31
7.3	Main components	31
7.4	Important <form> attributes	32
7.5	GET vs POST	33
7.6	Basic validation	33
7.7	Difference between <input> and <button>	34
7.8	Input Types in HTML5	34
8	Multimedia in HTML	39
8.1	What is multimedia?	39
8.2	Images with 	39
8.3	Audio with <audio>	40
8.4	Video with <video>	41
8.5	Subtitles and text tracks	41
8.6	iframes for external content	42
8.7	Multimedia best practices	42
9	Accessibility in HTML (WCAG and ARIA)	45
9.1	Why is accessibility important?	45
9.2	WCAG Guidelines (Web Content Accessibility Guidelines)	45
9.3	Correct HTML semantics	46
9.4	alt attributes on images	46
9.5	ARIA (Accessible Rich Internet Applications)	47
9.6	Keyboard navigation	48
9.7	Color contrast	48
9.8	Heading structure	49
9.9	Screen readers	49
9.10	Basic accessibility checklist	49
10	Advanced Attributes (Data Attributes and Global)	51
10.1	Data attributes (data-*)	51

10.2	Global attributes	52
10.3	Special attributes	55
10.4	Advanced attributes checklist	56
11	Advanced Meta Tags (Open Graph, Twitter Cards, PWA)	57
11.1	Open Graph (OG)	57
11.2	Twitter Cards	58
11.3	PWA (Progressive Web App)	59
11.4	Mobile meta tags	60
11.5	Complete SEO meta tags	61
11.6	Validate meta tags	61
11.7	Advanced meta tags checklist	62

Welcome to the **HTML Guide for Computer Engineering**.

This book will teach you the essential knowledge you need as a Computer Engineering student about **HTML (HyperText Markup Language)**. It's not an exhaustive manual, but a pragmatic introduction to the concepts and tools every computer engineer should master.

What is HTML?

HTML is the **standard markup language** for creating web pages. It provides the structure and meaning of web content, while CSS controls presentation and JavaScript adds interactivity.

As a computer engineer, understanding HTML is fundamental because:

- **It's the foundation of the web:** every web page, web application, and modern digital content is built on HTML.
- **It's simple yet powerful:** you can create structured content clearly and semantically.
- **It enables communication with other professionals:** designers, frontend developers, and accessibility specialists all speak the language of HTML.
- **It opens doors:** understanding HTML is the first step toward web development, progressive web apps (PWA), electronic documents, and beyond.

What will you learn?

In this book we will cover:

1. **Fundamental Concepts** — What tags are, attributes, and how an HTML document is structured.
2. **The HTML Document** — The anatomy of a page: DOCTYPE, head, body, meta tags, favicon, and SEO.
3. **Semantic Elements** — How to use the correct tags to give meaning to your content (headings, paragraphs, lists, citations).
4. **Links and Hyperlinks** — How to connect documents and resources with URLs, absolute and relative paths.
5. **Tables** — How to structure tabular data correctly.
6. **Your First HTML** — A practical project to bring it all together.

How to use this book

- Each chapter includes **practical examples** you can copy and experiment with.
- Concepts are ordered **from simple to complex**.
- It's recommended to follow the sequential order: each chapter builds on the previous ones.

Tools you'll need

- A **code editor** (VS Code, Sublime Text, Atom, etc.)
- A **web browser** (Chrome, Firefox, Safari, Edge)
- **Basic command line** (Git Bash, PowerShell, or Terminal)

That's all. HTML doesn't require complicated installations.

📖 Next books in the series

This book is the **first in a series** on fundamental web technologies:

1. ✓ **HTML Guide for Computer Engineering** (this book)
2. 📖 **CSS Guide for Computer Engineering** (coming soon)
3. 📖 **JavaScript Guide for Computer Engineering** (coming soon)

Each book can be used independently, but following the order is recommended for logical progression in web development learning.

Note for educators

If you're a teacher using this material for your course, you can:

- Adapt it to your curriculum
- Add your own exercises
- Create PDF versions for printing
- Publish it on your own web server

Consult the TeachBook documentation in the resources section.

Let's get started!

HTML FUNDAMENTAL CONCEPTS

In this section you will learn the basic concepts you need to understand to work with HTML. It's not complicated, but it's important to grasp these ideas well from the beginning.

1.1 What is a tag?

A **tag** is the basic unit of HTML. It's code that tells the browser how to display or interpret content.

A typical tag has this structure:

```
<tagname>content</tagname>
```

Example:

```
<p>This is a paragraph.</p>
```

Here, `<p>` is the **opening tag**, `This is a paragraph.` is the **content**, and `</p>` is the **closing tag**. The browser interprets this as a paragraph.

1.2 Self-closing tags

Some tags have no content and close themselves. They're called **empty tags** or **self-closing tags**:

```
  
<br>  
<input type="text">
```

These tags don't need a separate closing tag.

1.3 What are attributes?

Attributes are additional properties that add information to a tag. They're placed inside the opening tag.

```

```

In this example:

- `src` is the attribute that specifies the **path** to the image

- `alt` is the attribute that provides **alternative text** if the image can't be loaded

Attributes always have the form `name="value"`.

1.4 Nesting

Tags can contain other tags. This is called **nesting**. Example:

```
<div>
  <h1>Main Title</h1>
  <p>This paragraph is inside the div.</p>
</div>
```

Here, `<h1>` and `<p>` are **nested** inside the `<div>`. Order matters: the tag that opens first must close last.

1.5 Basic structure of an HTML document

Every HTML page has a minimal structure:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page title</title>
  </head>
  <body>
    <h1>Content here</h1>
  </body>
</html>
```

- `<!DOCTYPE html>` — tells the browser it's an HTML5 document
- `<html>` — the root tag that contains everything
- `<head>` — contains information about the page (not visible in the browser)
- `<body>` — contains the visible content of the page

1.6 Comments

Comments are annotations in the code that the browser ignores. They're useful for documenting your work:

```
<!-- This is a comment -->
<p>This text is visible</p>
<!-- You can explain here what this section does -->
```

Comments don't appear on the page, only in the source code.

1.7 Special characters

Sometimes you need to display special characters in HTML. Use **HTML entities**:

Character	Code	Use
<	<	Less than
>	>	Greater than
&	&	Ampersand
"	"	Double quotes
'	'	Single quotes
©	©	Copyright symbol
€	€	Euro symbol

Example:

```
<p>The price is &lt; 10€ and the formula is E &gt; mc2</p>
```

Following chapters will cover each of these concepts in depth.

1.8 Detailed Anatomy of Tags

Understanding the complete structure of a tag is fundamental for working correctly with HTML.

1.8.1 Parts of a tag

```
<p class="important" id="paragraph-1">This is content</p>
```

1. **Opening symbol** (<) — marks the start
2. **Tag name** (p) — defines what type of element it is
3. **Attributes** (class="important" id="paragraph-1") — additional information
4. **Closing symbol** (>) — ends the opening tag
5. **Content** (This is content) — what is displayed or processed
6. **Closing tag** (</p>) — marks the end of the element

1.8.2 Common tag types

Containers

Tags that contain other tags or content:

```
<div>...</div>      <!-- Generic block -->
<section>...</section> <!-- Semantic section -->
<article>...</article> <!-- Article -->
<nav>...</nav>      <!-- Navigation -->
<header>...</header> <!-- Header -->
```

(continues on next page)

(continued from previous page)

```
<footer>...</footer>    <!-- Footer -->
<main>...</main>    <!-- Main content -->
```

Text tags

Tags that format text:

```
<h1>...</h1>    <!-- Heading level 1 -->
<p>...</p>    <!-- Paragraph -->
<strong>...</strong>    <!-- Strong text (important) -->
<em>...</em>    <!-- Emphasis -->
<small>...</small>    <!-- Small text -->
<span>...</span>    <!-- Generic inline text -->
```

Empty (self-closing)

They don't contain content or a closing tag:

```

<input type="...">
<br>
<hr>
<meta name="..." content="...">
<link rel="..." href="...">
```

1.8.3 Hierarchy and correct nesting

Tags must be nested correctly:

```
<!-- ✓ CORRECT: Elements close in order -->
<div>
  <p>
    <strong>Important text</strong> in a paragraph
  </p>
</div>

<!-- ❌ INCORRECT: Tag overlap -->
<div>
  <p>
    <strong>Important text</p>
  </strong>
</div>
```

1.8.4 Common attributes in all tags

Some attributes work in almost any tag:

Attribute	Description	Example
id	Unique element identifier	id="section-1"
class	CSS class for styling	class="highlighted"
style	Inline CSS styles	style="color: red; "
title	Text that appears on hover	title="See more"
data-*	Custom data	data-number="42"

1.8.5 DOCTYPE and HTML declaration

The **DOCTYPE** is not really a tag, but it's mandatory at the beginning of the document:

```
<!DOCTYPE html>
```

This tells the browser to use the HTML5 specification. In earlier HTML versions there were several variants, but in HTML5 there's only one.

Immediately after comes the root tag:

```
<!DOCTYPE html>
<html lang="en">
  ...
</html>
```

The `lang` attribute specifies the main language of the document (important for accessibility and SEO).

1.8.6 HTML validation

To verify that your HTML is correct, you can use the official W3C validator:

1. Go to <https://validator.w3.org/>
2. Enter your code or URL
3. The validator will show you errors and warnings

Keeping HTML valid is important for:

- Browser compatibility
- Accessibility
- SEO (search engine ranking)
- Code maintainability

THE HTML DOCUMENT: INTERNAL STRUCTURE

Every HTML page has a specific internal structure. Although you only see the content of the `<body>` in the browser, there's much more happening in the document's head (`<head>`).

2.1 Complete structure of an HTML document

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Title of my page</title>
    <link rel="icon" href="favicon.ico">
    <meta name="description" content="Brief description">
  </head>
  <body>
    <h1>Visible content here</h1>
  </body>
</html>
```

2.2 What is the `<head>`?

The `<head>` is the **document header**. It contains:

- **Metadata** — information about the page
- **Links** — to stylesheets, fonts, icons
- **Title** — what appears in the browser tab
- **Scripts** — JavaScript code
- **Instructions** — how the browser should interpret the content

The content of the `<head>` **doesn't appear on the page**, only the `<body>` does.

2.3 The lang attribute

Always specify the main language:

```
<html lang="en">  <!-- English -->
<html lang="es">  <!-- Spanish -->
<html lang="fr">  <!-- French -->
```

This is important for:

- Search engines (SEO)
- Screen readers (accessibility)
- Browser spell checkers

2.4 Meta charset

Specifies the **character encoding** of the document:

```
<meta charset="UTF-8">
```

UTF-8 is the modern standard and supports all languages and symbols. **Always use UTF-8.**

2.5 Meta viewport

Makes the page **responsive** (adapts to mobile screens):

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Without this, your page would look bad on mobile devices.

2.6 Page title

```
<title>My First Website</title>
```

The title appears in:

- The browser tab
- Search results
- Browser history

Important rule: each page should have a unique and descriptive title.

2.7 Favicon

The **favicon** is the small icon that appears in the browser tab:

```
<link rel="icon" href="favicon.ico">
```

You can use:

- `favicon.ico` — traditional format
- `favicon.png` — modern format
- `favicon.svg` — vector (recommended)

2.8 Meta description

A brief description of your page that appears in search results:

```
<meta name="description" content="Learn HTML from scratch. A practical guide for  
computer engineers.">
```

It should be 150-160 characters to appear complete in Google.

2.9 Meta tags for social media

Control how your page appears when shared on social media:

```
<meta property="og:title" content="My First Website">  
<meta property="og:description" content="Brief description">  
<meta property="og:image" content="image.png">  
<meta property="og:url" content="https://example.com">
```

These are **Open Graph** tags used by Facebook, Twitter, LinkedIn, etc.

2.10 Linking stylesheets

```
<link rel="stylesheet" href="styles.css">
```

By convention, CSS links go in the `<head>`.

2.11 Scripts

```
<script src="script.js"></script>
```

Although they can technically go in the `<head>` or at the end of the `<body>`, they're usually placed at the end of the `<body>` so the page loads faster.

The following chapters will dive deeper into each of these elements.

2.12 Charset (Character Encoding)

The `charset` attribute defines which **character encoding** your HTML document uses. It's essential for browsers to correctly interpret text in different languages and special characters.

2.12.1 What is charset?

The charset specifies the **character encoding** used by the file:

- **UTF-8** — Universal charset (recommended, includes all languages)
- **ISO-8859-1** — Latin characters
- **Windows-1252** — Windows characters
- **ASCII** — Basic English characters

2.12.2 Declare charset in <head>

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Page</title>
</head>
<body>
  <h1>Hello, world!</h1>
</body>
</html>
```

Important rule: The `<meta charset>` must be the **first element** inside `<head>` so the browser processes it immediately.

```
<!-- ✓ CORRECT: Meta charset is first -->
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Page</title>
</head>

<!-- ❌ INCORRECT: Meta charset is not first -->
<head>
  <title>Page</title>
  <meta charset="UTF-8">
</head>
```

2.12.3 UTF-8: The universal standard

UTF-8 is the default and recommended charset:

✓ Supports **all world languages** ✓ Compatible with **special characters** ✓ Efficient in **file size** ✓ Modern web standard

Examples of supported Unicode characters

```
<meta charset="UTF-8">

<!-- Spanish -->
<p>Accents: á, é, í, ó, ú, ñ</p>

<!-- French -->
<p>Characters: ç, è, ê, ë</p>

<!-- German -->
<p>Umlauts: ä, ö, ü, ß</p>

<!-- Mathematical characters and symbols -->
<p>Symbols: © ® ™ € ¥ ± × ÷ ≠ ≤ ≥</p>

<!-- Emojis and special symbols supported -->
<p>Unicode supports many special characters</p>
```

2.12.4 Common problems without correct charset

Without declaring charset="UTF-8", the browser may misinterpret characters:

```
<!-- ❌ BAD: Without meta charset, accented characters may appear as:
    broken characters instead of "Hello"
-->
<head>
  <title>Page without charset</title>
</head>

<!-- ✓ GOOD: With charset, everything displays correctly -->
<head>
  <meta charset="UTF-8">
  <title>Correct page</title>
</head>
```

2.12.5 HTML entities for special characters

If for some reason you need special characters without UTF-8, use **HTML entities**:

Character	Entity	Description
©	©	Copyright
®	®	Registered trademark
™	™	Trademark
€	€	Euro
¥	¥	Yen
£	£	Pound
§	§	Section
¶	¶	Paragraph
†	†	Dagger
‡	‡	Double dagger

Example:

```
<p>Copyright &copy; 2026 University of Salamanca</p>
<!-- Will display as: Copyright © 2026 University of Salamanca -->
```

2.12.6 Save files as UTF-8

Make sure your **text editor** saves files in **UTF-8** encoding:

VS Code

1. Click UTF-8 in the bottom right corner
2. Select Save with Encoding... → UTF-8

Other editors

- **Notepad++** — Encoding → Encode in UTF-8
- **Sublime Text** — File → Save with Encoding → UTF-8
- **Atom** — File → Save As → Encoding: UTF-8

2.12.7 Charset checklist

- ✓ `<meta charset="UTF-8">` is the first element in `<head>`
- ✓ File saved as **UTF-8** in your editor
- ✓ Accented characters display correctly in browser
- ✓ Emoji and special symbols work
- ✓ Don't use obsolete charsets (ISO-8859-1, Windows-1252)

With charset properly configured, your page will display any language and special character correctly. In the next topic you'll learn about other important meta attributes.

HTML SEMANTIC ELEMENTS

HTML semantics means using tags that correctly describe the meaning of their content. It's not just about how it looks in the browser, but about what that content *means*.

3.1 Why does semantics matter?

Using semantic HTML is important for:

1. **Search engines** — understand your content better for SEO
2. **Screen readers** — helps people with visual disabilities
3. **Maintainability** — your code is easier to understand and modify
4. **Accessibility** — you meet important web standards

3.2 Example: Semantic vs. Non-semantic

Incorrect (non-semantic):

```
<div>
  <div><b>My Article</b></div>
  <div>Date: June 15</div>
  <div>The article content goes here...</div>
</div>
```

Correct (semantic):

```
<article>
  <h1>My Article</h1>
  <time datetime="2025-06-15">June 15</time>
  <p>The article content goes here...</p>
</article>
```

3.3 Main semantic tags

3.3.1 General structure

- `<header>` — header of the page or section
- `<nav>` — navigation menu
- `<main>` — unique main content (only one per page)
- `<section>` — thematic section
- `<article>` — independent content (blog post, news, etc.)
- `<aside>` — sidebar or related content
- `<footer>` — page footer

3.3.2 Text and content

- `<h1>` to `<h6>` — headings (h1 is most important)
- `<p>` — paragraph
- `<strong>` — important text (not just bold)
- `<em>` — emphasis (not just italic)
- `<mark>` — highlighted text
- `<small>` — small text (notes, copyright)
- `<code>` — programming code
- `<pre>` — preformatted text

3.3.3 Lists

- `<ul>` — unordered list (bullets)
- `<ol>` — ordered list (numbers)
- `<li>` — list item
- `<dl>` — definition list
- `<dt>` — term to define
- `<dd>` — definition of term

3.3.4 Quotes and references

- `<blockquote>` — long quote
- `<q>` — short inline quote
- `<cite>` — reference to a work
- `<abbr>` — abbreviation

3.4 Structure of a semantic HTML page

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>My Blog</title>
</head>
<body>
  <header>
    <h1>My Website</h1>
    <nav>
      <a href="/">Home</a>
      <a href="/blog">Blog</a>
      <a href="/contact">Contact</a>
    </nav>
  </header>

  <main>
    <article>
      <h2>My First Article</h2>
      <p>Content...</p>
    </article>
  </main>

  <aside>
    <h3>Recent Articles</h3>
    <ul>
      <li><a href="#">Article 1</a></li>
      <li><a href="#">Article 2</a></li>
    </ul>
  </aside>

  <footer>
    <p>&copy; 2025 My Website</p>
  </footer>
</body>
</html>
```

3.5 Difference between `<strong>` and `<b>`

- `<strong>` — **important** text (semantic)
- `<b>` — **bold** text (presentation only)

Same with `<em>` vs `<i>`.

3.6 Accessibility and semantics

When you use semantic HTML:

- Screen readers understand the structure
- Search engines index your content better
- Your page is easier to maintain
- You comply with WCAG standards

Following chapters will show you how to use each of these tags in detail.

LINKS AND HYPERLINKS

Links are the essence of the web. Without them, there would be no way to navigate from one page to another. In this section you'll learn how to create and manage links correctly.

4.1 The <a> tag

The <a> tag (anchor) creates a hyperlink. Its main attribute is href (Hypertext REFerence):

```
<a href="https://example.com">Click here</a>
```

The text between <a> and is what the user sees and can click on.

4.2 URLs vs. Paths

There are two ways to specify destinations:

4.2.1 Absolute URLs

Complete web address:

```
<a href="https://www.google.com">Google</a>  
<a href="http://example.com/page.html">External page</a>
```

4.2.2 Relative Paths

Paths within your own site:

```
<a href="page.html">Page in same directory</a>  
<a href="folder/page.html">Page in a folder</a>  
<a href="../page.html">Page in parent folder</a>  
<a href="/">Home page</a>
```

4.3 Link attributes

Attribute	Use	Example
href	Link destination	href="page.html"
target	Where to open	target="_blank" (new tab)
title	Text on hover	title="See more information"
rel	Relation to destination	rel="noopener noreferrer"
download	Download file	download="file.pdf"

4.4 Open in new tab

```
<a href="https://example.com" target="_blank" rel="noopener noreferrer">  
  Open in new tab  
</a>
```

The `rel="noopener noreferrer"` attribute is important for security.

4.5 Internal links within the page

Use `id` to create links to sections:

```
<!-- Create a destination -->  
<h2 id="section-2">Section 2</h2>  
  
<!-- Create a link to that destination -->  
<a href="#section-2">Go to Section 2</a>
```

4.6 Download files

```
<a href="document.pdf" download>Download PDF</a>  
<a href="image.png" download="my-image.png">Download with different name</a>
```

4.7 Email and phone

```
<!-- Email link -->  
<a href="mailto:user@example.com">Send email</a>  
  
<!-- Phone link -->  
<a href="tel:+34123456789">Call</a>  
  
<!-- Email with parameters (add ?subject=topic to href) -->  
<a href="mailto:user@example.com">Send inquiry</a>
```

4.8 Best practices

1. **Descriptive text** — use text that explains where the link goes, not “click here”
2. **Clear structure** — links should be easy to identify
3. **URL validation** — verify that links work
4. **Accessibility** — links must be navigable with keyboard

4.9 Difference between paths

Imagine this structure:

```
site/
├── index.html
├── page1.html
└── folder/
    └── page2.html
```

From `folder/page2.html`:

- `../index.html` → go up one folder, access `index.html`
- `../page1.html` → go up one folder, access `page1.html`
- `./file.html` → in current folder
- `../../file.html` → go up two folders

The following chapters will teach you more about paths and how to organize them correctly.

TABLES IN HTML

Tables are fundamental elements for presenting tabular data in a structured way. Although they were historically used for page layouts (we use CSS for that today), tables remain essential for real data.

5.1 Basic table structure

```
<table>
  <tr>
    <td>Cell 1</td>
    <td>Cell 2</td>
  </tr>
  <tr>
    <td>Cell 3</td>
    <td>Cell 4</td>
  </tr>
</table>
```

5.2 Table tags

Tag	Meaning	Use
<table>	Defines the table	Main container
<tr>	Table Row	Each row of data
<td>	Table Data	Normal cells
<th>	Table Header	Header cells
<thead>	Table Head	Groups headers
<tbody>	Table Body	Groups data body
<tfoot>	Table Footer	Groups table footer
<caption>	Caption	Table title
<colgroup>	Column Group	Groups columns
<col>	Column	Defines a column

5.3 Table with complete structure

```
<table>
  <caption>Monthly Temperatures 2025</caption>
  <thead>
    <tr>
      <th>Month</th>
      <th>Min (°C)</th>
      <th>Max (°C)</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>January</td>
      <td>5</td>
      <td>15</td>
    </tr>
    <tr>
      <td>February</td>
      <td>6</td>
      <td>16</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td>Average</td>
      <td>5.5</td>
      <td>15.5</td>
    </tr>
  </tfoot>
</table>
```

5.4 Cell attributes

Attribute	Use	Example
colspan	Merge columns	<td colspan="2">
rowspan	Merge rows	<td rowspan="3">
headers	Associate to headers	<td headers="col1 col2">
scope	Header scope	<th scope="col">

5.5 Merged cells

```
<table>
  <tr>
    <th>Name</th>
    <th colspan="2">Grades</th>
  </tr>
  <tr>
    <td>John</td>
```

(continues on next page)

(continued from previous page)

```
<td>8.5</td>
<td>9.0</td>
</tr>
</table>
```

5.6 Accessibility in tables

To make tables accessible for screen readers:

```
<table>
  <thead>
    <tr>
      <th scope="col" id="month">Month</th>
      <th scope="col" id="temp">Temperature</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td headers="month">January</td>
      <td headers="temp">15°C</td>
    </tr>
  </tbody>
</table>
```

5.7 Complex tables

Complex tables with multiple levels of headers can be hard to make accessible. In those cases, consider:

1. **Simplify the table** if possible
2. **Split it into simpler tables**
3. **Use accessibility attributes** like `scope` and `headers`
4. **Provide alternatives** (descriptive text, list, etc.)

5.8 When to use tables?

Use them for:

- Real tabular data
- Comparisons
- Statistics
- Schedules

DON'T use them for:

- Page layouts (use CSS Grid or Flexbox)
- Non-tabular content

- Visual effects

In upcoming chapters you'll learn to style tables with CSS to make them more visually appealing.

YOUR FIRST HTML PROJECT

Now that you've learned the fundamentals, it's time to create your first real web page. We'll create a simple but complete website that combines everything we've covered.

6.1 Project Objective

You'll create a **personal website** that includes:

- Correct HTML5 structure
- Multiple semantic sections
- Functional navigation
- Properly formatted content
- Internal and external links

6.2 Project Structure

```
my-website/
├── index.html
├── about-me.html
├── projects.html
├── contact.html
├── css/
│   └── styles.css
```

6.3 The index.html file

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Personal Page - John Developer</title>
  <meta name="description" content="Welcome to my personal page. I'm a web developer,
  passionate about technology.">
  <link rel="stylesheet" href="css/styles.css">
```

(continues on next page)

(continued from previous page)

```

</head>
<body>
  <!-- Header and navigation -->
  <header>
    <h1>John Developer</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="about-me.html">About Me</a>
      <a href="projects.html">Projects</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <!-- Main content -->
  <main>
    <section id="introduction">
      <h2>Welcome</h2>
      <p>Hello, I'm <strong>John</strong>, a Computer Engineering student passionate
about web development.</p>
      <p>On this site you'll find information about me and my projects.</p>
    </section>

    <section id="highlights">
      <h2>Highlights</h2>
      <ul>
        <li>Programming in HTML, CSS and JavaScript</li>
        <li>Web application development</li>
        <li>Responsive design</li>
      </ul>
    </section>

    <section id="knowledge">
      <h2>My Knowledge</h2>
      <table>
        <thead>
          <tr>
            <th>Language</th>
            <th>Level</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>HTML</td>
            <td>Intermediate</td>
          </tr>
          <tr>
            <td>CSS</td>
            <td>Basic</td>
          </tr>
          <tr>
            <td>JavaScript</td>
            <td>Basic</td>
          </tr>
        </tbody>
      </table>
    </section>
  </main>

```

(continues on next page)

(continued from previous page)

```

<!-- Sidebar (optional) -->
<aside>
  <h3>Useful Links</h3>
  <ul>
    <li><a href="https://github.com" target="_blank" rel="noopener noreferrer">My
↳ GitHub</a></li>
    <li><a href="https://linkedin.com" target="_blank" rel="noopener noreferrer">My
↳ LinkedIn</a></li>
    <li><a href="mailto:john@example.com">Email me</a></li>
  </ul>
</aside>

<!-- Footer -->
<footer>
  <p>&copy; 2025 John Developer. All rights reserved.</p>
  <p><small>Last updated: June 2025</small></p>
</footer>
</body>
</html>

```

6.4 Other pages

6.4.1 about-me.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>About Me - John Developer</title>
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>
  <header>
    <h1>John Developer</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="about-me.html">About Me</a>
      <a href="projects.html">Projects</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <main>
    <article>
      <h2>About Me</h2>
      <p>I'm a Computer Engineering student at the University...</p>
      <h3>My Journey</h3>
      <p>I started learning programming 3 years ago...</p>
    </article>
  </main>

```

(continues on next page)

(continued from previous page)

```
<footer>
  <p>&copy; 2025 John Developer.</p>
</footer>
</body>
</html>
```

6.5 Steps to create your project

1. **Create a folder** called `my-website`
2. **Copy the code** from `index.html` into a new file
3. **Open the page** in your browser (File → Open)
4. **Create the other pages** (`about-me.html`, `projects.html`, `contact.html`)
5. **Verify that links** work between pages
6. **Validate your HTML** at validator.w3.org

6.6 Next steps

Once you have the basic HTML working:

1. **Learn CSS** to style your page
2. **Add JavaScript** for interactivity
3. **Publish your site** on GitHub Pages or a web server
4. **Keep improving** your project

6.7 Review Checklist

- ☐ All files are in UTF-8
- ☐ You have a correct DOCTYPE
- ☐ Navigation works between pages
- ☐ External links open in new tab (with `target="_blank"`)
- ☐ Your HTML validates without errors
- ☐ Relative paths work correctly
- ☐ You used semantic tags (`<header>`, `<main>`, `<footer>`, etc.)

Congratulations! You've created your first website. Now it's time to style it and make it more interesting.

FORMS IN HTML

Forms are one of the most important HTML elements. They allow users to interact with your website by sending data to the server. Everything from login to searches goes through forms.

7.1 Why are forms important?

Forms are the primary way to:

- ✓ Collect user information
- ✓ Process data (login, registration, search)
- ✓ Enable bidirectional interaction
- ✓ Improve user experience
- ✓ Validate data before sending to server

7.2 Basic form structure

A simple form has this structure:

```
<form action="/process" method="POST">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required>

  <button type="submit">Send</button>
</form>
```

7.3 Main components

A complete form includes:

7.3.1 1. The <form> tag

```
<form action="/process" method="POST">
  <!-- Form content -->
</form>
```

- **action** — URL where data is sent
- **method** — HTTP method: GET or POST

7.3.2 2. Input fields

```
<input type="text" name="username">
<textarea name="message"></textarea>
<select name="country">
  <option>Spain</option>
</select>
```

7.3.3 3. <label> tags

```
<label for="email">Email:</label>
<input type="email" id="email" name="email">
```

- Make the form accessible
- Increase clickable area
- Describe the field

7.3.4 4. Buttons

```
<button type="submit">Send</button>
<button type="reset">Clear</button>
<button type="button">Action</button>
```

7.4 Important <form> attributes

Attribute	Use	Example
action	Destination URL	action="/login"
method	GET or POST	method="POST"
enctype	Encoding (for files)	enctype="multipart/form-data"
name	Form name	name="login-form"
autocomplete	Autocomplete	autocomplete="on"
novalidate	No HTML5 validation	novalidate

7.5 GET vs POST

7.5.1 GET

- Sends data in the URL
- Visible in address bar
- Limited to ~2000 characters
- Use: searches, filters

```
<form action="/search" method="GET">
  <input type="search" name="q">
  <button>Search</button>
</form>
```

7.5.2 POST

- Sends data in request body
- Not visible in URL
- No character limit
- Use: login, registration, sensitive data

```
<form action="/login" method="POST">
  <input type="email" name="email">
  <input type="password" name="password">
  <button>Sign In</button>
</form>
```

7.6 Basic validation

HTML5 allows native validation without JavaScript:

```
<form>
  <input type="email" required>      <!-- Required -->
  <input type="url" required>        <!-- Must be URL -->
  <input type="number" min="1" max="100"> <!-- Numeric range -->
  <input type="text" pattern="[A-Z]{3}"> <!-- Custom pattern -->
  <button type="submit">Send</button>
</form>
```

7.7 Difference between `<input>` and `<button>`

```
<!-- Input: only submits data -->
<input type="submit" value="Send">

<!-- Button: more flexible -->
<button type="submit">Send</button>
```

`<button>` is recommended because:

- ✓ Allows HTML content inside
- ✓ Easier to style
- ✓ Better accessibility

In upcoming topics you'll learn:

1. **Input types** — Email, password, date, etc.
2. **Validation** — HTML5 validation rules
3. **Advanced elements** — Textarea, select, datalist
4. **Accessibility** — Making forms accessible
5. **Best practices** — Security and UX

7.8 Input Types in HTML5

HTML5 provides many specialized `<input>` types that offer automatic validation and better user experience. Each type has a specific purpose.

7.8.1 Most common input types

1. Text (Plain text)

```
<input type="text" name="name" placeholder="Your name">
```

- Single line plain text
- Use: names, addresses, etc.

2. Email

```
<input type="email" name="email" required>
```

- Validates email format automatically
- Shows email keyboard on mobile (@)
- Use: registration, login

3. Password

```
<input type="password" name="pass" required>
```

- Hides text while typing
- Browsers can save/retrieve
- Use: login, registration

4. Number

```
<input type="number" name="age" min="18" max="120">
```

- Numbers only
- Range validation (min/max)
- Shows numeric keyboard on mobile
- Attributes: min, max, step

5. Tel (Telephone)

```
<input type="tel" name="phone" pattern="[0-9]{9}">
```

- Shows phone keyboard on mobile
- Doesn't validate format automatically
- Use: phone numbers

6. URL

```
<input type="url" name="website" required>
```

- Validates valid URL format
- Use: websites, links

7. Date

```
<input type="date" name="birthdate">
```

- Date picker selector
- Opens date picker in modern browsers
- Format: YYYY-MM-DD
- Attributes: min, max

8. Time

```
<input type="time" name="time">
```

- Time picker
- Format: HH:MM
- Use: schedules, bookings

9. Search

```
<input type="search" name="q" placeholder="Search...">
```

- Optimized for searches
- Some browsers show clear button (X)
- Use: search bars

10. Range

```
<input type="range" name="volume" min="0" max="100" value="50">
```

- Slider control
- Value between min and max
- Use: volume, brightness, zoom

11. Checkbox

```
<label>  
  <input type="checkbox" name="accept" value="yes">  
  I accept the terms  
</label>
```

- Select/deselect
- Can be checked or not
- Attribute: checked

12. Radio

```
<label>  
  <input type="radio" name="gender" value="male"> Male  
</label>  
<label>  
  <input type="radio" name="gender" value="female"> Female  
</label>
```

- Only one option can be selected
- Same name groups the options

- Use: choose one from several

13. File

```
<input type="file" name="document" accept=".pdf,.doc">
```

- Opens file explorer
- accept attribute filters extensions
- For multiple: multiple

14. Color

```
<input type="color" name="color" value="#FF0000">
```

- Color picker
- Returns hexadecimal value
- Use: custom themes

15. Hidden

```
<input type="hidden" name="token" value="abc123">
```

- Not shown to user
- Submitted with the form
- Use: tokens, internal IDs

7.8.2 Quick reference table

Type	Validation	Mobile Keyboard	Use
text	No	Text	General
email	Yes	Email (@)	Emails
password	No	Text	Passwords
number	Yes	Numeric	Numbers
tel	No	Phone	Phones
url	Yes	URL (.)	URLs
date	Yes	Date picker	Dates
time	No	Time	Times
search	No	Search	Searches
range	No	Slider	Ranges
checkbox	No	-	Multiple
radio	No	-	One option
file	No	-	Files
color	No	-	Colors
hidden	No	-	Hidden data

7.8.3 Common attributes for inputs

```
<input
  type="text"
  name="username"           <!-- Field name -->
  id="username"            <!-- Unique ID -->
  value="john"             <!-- Initial value -->
  placeholder="Your name"  <!-- Hint -->
  required                 <!-- Required -->
  disabled                 <!-- Disabled -->
  readonly                 <!-- Read-only -->
  autocomplete="on"        <!-- Autocomplete -->
  maxlength="50"           <!-- Max characters -->
  minlength="3"            <!-- Min characters -->
>
```

7.8.4 Best practice: Always use <label>

```
<!-- ✓ CORRECT -->
<label for="email">Email:</label>
<input type="email" id="email" name="email">

<!-- ✗ INCORRECT -->
Email: <input type="email" name="email">
```

Labels:

- ✓ Make the form accessible
- ✓ Increase clickable area
- ✓ Improve user experience

MULTIMEDIA IN HTML

Multimedia elements allow you to integrate images, audio, and video directly into your HTML pages. HTML5 provides native tags without needing external plugins.

8.1 What is multimedia?

Multimedia refers to:

-  **Images** — PNG, JPG, SVG, WebP
-  **Audio** — MP3, OGG, WAV
-  **Video** — MP4, WebM, OGG
-  **Embedded content** — iframes, maps

8.2 Images with `<img>`

The most basic tag:

```

```

Important attributes:

- `src` — Image path (required)
- `alt` — Alternative text (required for accessibility)
- `width/height` — Dimensions
- `loading="lazy"` — Lazy loading

8.2.1 Responsive images

To adapt images to different screens:

```

```

- `srcset` — List of images by width
- `sizes` — Which image to use per viewport
- Browser automatically chooses

8.2.2 `<picture>` element for full control

```
<picture>
  <source media="(min-width: 1200px)" srcset="image-large.jpg">
  <source media="(min-width: 600px)" srcset="image-medium.jpg">
  
</picture>
```

Use: different images per device or format.

8.3 Audio with `<audio>`

Native audio playback:

```
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
  <source src="song.ogg" type="audio/ogg">
  Your browser doesn't support HTML5 audio.
</audio>
```

Attributes:

- `controls` — Shows playback controls
- `autoplay` — Plays automatically
- `loop` — Continuous repeat
- `muted` — Muted by default
- `preload="auto|metadata|none"` — How to load

Formats:

- MP3 — Compatible with all
- OGG — Open source alternative

- WAV — High quality

8.4 Video with <video>

Native video playback:

```
<video controls width="640" height="360">
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  Your browser doesn't support HTML5 video.
</video>
```

Attributes:

- controls — Playback controls
- poster="image.jpg" — Preview image
- autoplay — Plays automatically
- loop — Repeat
- muted — Muted
- preload — How to load

Formats:

- MP4 — Best compatibility
- WebM — Better compression
- OGG — Open source alternative

8.5 Subtitles and text tracks

```
<video controls>
  <source src="video.mp4">
  <track kind="subtitles" src="subtitles-en.vtt" srclang="en" label="English">
  <track kind="subtitles" src="subtitles-es.vtt" srclang="es" label="Español">
  <track kind="captions" src="captions.vtt" srclang="en">
</video>
```

Track types:

- subtitles — Dialogue translation
- captions — Full transcript (including sounds)
- descriptions — Description for visually impaired

8.6 iframes for external content

```
<iframe
  src="https://www.youtube.com/embed/VIDEO_ID"
  width="560"
  height="315"
  frameborder="0"
  allowfullscreen>
</iframe>
```

Security attributes:

- `sandbox` — Security restrictions
- `allow` — Specific permissions

Secure example:

```
<iframe
  src="https://www.youtube.com/embed/VIDEO_ID"
  sandbox="allow-playback allow-same-origin"
  title="YouTube Video">
</iframe>
```

8.7 Multimedia best practices

8.7.1 Make multimedia accessible

```

<audio controls>
  <source src="podcast.mp3">
  <p>Your browser doesn't support audio.</p>
</audio>
```

8.7.2 Use multiple formats

```
<video controls>
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
</video>
```

8.7.3 Optimize images

- Use responsive `srcset` tool
- Compress without losing quality
- Consider WebP as modern alternative

8.7.4 ? Lazy loading

```

```

8.7.5 ? Avoid

- Audio/video auto-playing without permission
- Images without `alt` attribute
- Multimedia without text alternatives

In upcoming topics you'll learn:

1. Responsive images in detail
2. Multimedia optimization
3. JavaScript integration
4. Accessibility in multimedia

ACCESSIBILITY IN HTML (WCAG AND ARIA)

Web accessibility means creating websites that everyone can use, including people with disabilities. It's a right, a legal responsibility, and good engineering practice.

9.1 Why is accessibility important?

1. **Inclusion** — Benefits ~15% of global population with disabilities
2. **Legal** — Many countries require accessibility (ADA in USA, AODA in Canada)
3. **SEO** — Google rewards accessible sites
4. **Business** — Reaches more users
5. **Quality** — More semantic and maintainable code

9.2 WCAG Guidelines (Web Content Accessibility Guidelines)

WCAG 2.1 defines three levels:

- **A** — Basic compliance
- **AA** — Intermediate compliance (recommended)
- **AAA** — Maximum compliance

Core principles (POUR):

1. **Perceptible** — Information perceivable
2. **Operable** — Keyboard navigation
3. **Understandable** — Understandable content
4. **Robust** — Compatible with assistive technologies

9.3 Correct HTML semantics

The first step is using **proper HTML tags**:

```
<!-- ✓ CORRECT: Semantic -->
<header>
  <nav>
    <ul>
      <li><a href="/home">Home</a></li>
      <li><a href="/about">About</a></li>
    </ul>
  </nav>
</header>

<main>
  <article>
    <h1>Article title</h1>
    <p>Content...</p>
  </article>
</main>

<footer>
  <p>&copy; 2026</p>
</footer>

<!-- ❌ INCORRECT: Generic divs -->
<div class="header">
  <div class="nav">
    <div class="menu">...</div>
  </div>
</div>
```

Important semantic tags:

- <header>, <nav>, <main>, <article>, <section>, <aside>, <footer>
- <h1> to <h6> for headings (in order)
- <label> for form fields
- <table> for tabular data

9.4 alt attributes on images

The alt attribute is **required** and critical:

```
<!-- ✓ GOOD: Meaningful description -->


<!-- ✓ ACCEPTABLE: Decorative image -->


<!-- ❌ BAD: Missing alt -->


<!-- ❌ BAD: Redundant -->
 <!-- Not useful -->
```

Rule: If image communicates information, describe what it says. If purely decorative, use `alt=""`.

9.5 ARIA (Accessible Rich Internet Applications)

ARIA provides **attributes** to describe role, state, and properties:

9.5.1 ARIA Roles

```
<!-- Define element role -->
<div role="button" onclick="doSomething()">Click here</div>

<div role="alert">Important warning!</div>

<div role="navigation">...</div>

<div role="search">...</div>
```

9.5.2 Labels and descriptions

```
<!-- aria-label: invisible label -->
<button aria-label="Close menu">x</button>

<!-- aria-labelledby: reference another element -->
<h2 id="dialog-title">Confirm action</h2>
<div role="dialog" aria-labelledby="dialog-title">
  Are you sure?
</div>

<!-- aria-describedby: additional description -->
<input type="password" aria-describedby="pwd-hint">
<p id="pwd-hint">Minimum 8 characters, include numbers</p>
```

9.5.3 State properties

```
<!-- aria-expanded: expanded/collapsed -->
<button aria-expanded="false" aria-controls="menu">Menu</button>
<div id="menu" hidden>Options...</div>

<!-- aria-current: current page -->
<nav>
  <a href="/home" aria-current="page">Home</a>
  <a href="/about">About</a>
</nav>

<!-- aria-disabled: disabled -->
<button aria-disabled="true">Not available</button>

<!-- aria-live: live updates -->
<div aria-live="polite" aria-atomic="true">
```

(continues on next page)

```
New messages loaded...  
</div>
```

9.6 Keyboard navigation

9.6.1 Tab order

```
<!-- Use tabindex=0 for non-standard interactive elements -->  
<div tabindex="0" role="button">Clickable</div>  
  
<!-- Avoid positive tabindex (breaks natural order) -->  
<!-- ❌ BAD: tabindex="1", tabindex="2", etc. -->  
  
<!-- To remove from keyboard navigation -->  
<div tabindex="-1">Not keyboard navigable</div>
```

9.6.2 Focus visibility

```
<!-- Ensure focus is visible -->  
<style>  
  button:focus {  
    outline: 2px solid blue;  
    outline-offset: 2px;  
  }  
</style>  
  
<!-- ❌ BAD: Remove outline without alternative -->  
button:focus {  
  outline: none; /* Never do this alone */  
}
```

9.7 Color contrast

WCAG AA requires:

- **Normal:** ratio 4.5:1
- **Large:** ratio 3:1

```
<!-- ✓ GOOD: Sufficient contrast -->  
<p style="color: #000; background: #fff;">High contrast text</p>  
  
<!-- ❌ BAD: Insufficient contrast -->  
<p style="color: #ccc; background: #ddd;">Hard to read</p>
```

9.8 Heading structure

```
<!-- ✓ CORRECT: Logical hierarchy -->
<h1>Main page title</h1>

<section>
  <h2>Section 1</h2>
  <h3>Subsection 1.1</h3>
  <h3>Subsection 1.2</h3>
</section>

<section>
  <h2>Section 2</h2>
</section>

<!-- ❌ INCORRECT: Skipped levels -->
<h1>Title</h1>
<h3>Direct jump to h3</h3> <!-- Confusing for screen readers -->
```

9.9 Screen readers

Blind/visually impaired users use readers like:

- NVDA (free, Windows)
- JAWS (Windows)
- VoiceOver (macOS/iOS)
- TalkBack (Android)

Tip: test your site with NVDA to verify accessibility.

9.10 Basic accessibility checklist

- ✓ Use semantic HTML tags
- ✓ All visual content has `alt` attribute
- ✓ Forms have `<label>` elements
- ✓ Color contrast $\geq 4.5:1$
- ✓ Keyboard navigable (Tab, Enter, Escape)
- ✓ Focus visible on all interactive elements
- ✓ Headings in hierarchical order
- ✓ Videos have captions and descriptions
- ✓ Links have descriptive text (not “click here”)
- ✓ Site works without JavaScript

In upcoming topics you’ll learn:

1. ARIA in depth
2. Accessibility testing
3. Audit tools
4. Common accessible patterns

ADVANCED ATTRIBUTES (DATA ATTRIBUTES AND GLOBAL)

Advanced attributes allow you to store custom data, control special behaviors, and enhance page interactivity without complex JavaScript.

10.1 Data attributes (data-*)

Data attributes let you **store custom information** in HTML elements, accessible from JavaScript:

```
<!-- Element with custom data -->
<div class="product"
    data-id="12345"
    data-price="29.99"
    data-available="true">
  Gaming Laptop
</div>

<button data-action="save" data-confirmation="true">
  Save changes
</button>
```

10.1.1 Access from JavaScript

```
// Get data
const product = document.querySelector('.product');
console.log(product.dataset.id);           // "12345"
console.log(product.dataset.price);        // "29.99"
console.log(product.dataset.available);    // "true"

// Set data
product.dataset.status = 'sold';
```

10.1.2 Common use cases

```
<!-- E-commerce: product information -->
<div class="cart-item"
    data-sku="PROD-001"
    data-quantity="2"
    data-total="59.98">
    <h3>Product A</h3>
    <p>$29.99 x 2</p>
</div>

<!-- Forms: custom validation -->
<input type="text"
    data-validate="email"
    data-required="true"
    placeholder="Your email">

<!-- Galleries: image information -->


<!-- Animations: configuration -->
<div class="animation"
    data-duration="2000"
    data-delay="500"
    data-repeat="infinite">
    Animated content
</div>
```

10.2 Global attributes

Global attributes work on any HTML element:

10.2.1 id — Unique identifier

```
<!-- Define unique identifier for element -->
<h1 id="main-title">Welcome</h1>

<!-- Use in internal links -->
<a href="#main-title">Go to title</a>

<!-- Use in JavaScript -->
<button onclick="document.getElementById('main-title').style.color='red';">
    Change color
</button>
```

Rules:

- ✓ Unique identifier per page
- ✓ No spaces or special characters

- ✓ Start with letter or underscore

10.2.2 class — CSS classes

```
<!-- Apply multiple classes -->
<div class="container main large">
  Content with multiple styles
</div>

<!-- CSS -->
<style>
  .container { padding: 20px; }
  .main { font-weight: bold; }
  .large { width: 100%; }
</style>
```

10.2.3 style — Inline styles (not recommended)

```
<!-- Direct styles (avoid when possible) -->
<p style="color: blue; font-size: 18px;">
  Paragraph with inline styles
</p>

<!-- ✓ BETTER: Use classes in external CSS -->
<p class="highlighted-text">
  Paragraph with class
</p>
```

10.2.4 title — Information on hover

```
<!-- Tooltip on hover -->
<button title="Click to save">Save</button>



<abbr title="General Assembly">GA</abbr>
```

10.2.5 hidden — Hide elements

```
<!-- Hide element without removing from DOM -->
<div hidden>
  Hidden content (invisible but exists in HTML)
</div>

<!-- Show with JavaScript -->
<button onclick="document.querySelector('div').hidden = false;">
  Show
</button>
```

10.2.6 tabindex — Keyboard navigation order

```
<!-- Element focusable by keyboard -->
<div tabindex="0" role="button">Interactive element</div>

<!-- Elements not keyboard navigable (e.g., decorative) -->
<span tabindex="-1">Non-navigable element</span>

<!-- AVOID: Positive order breaks natural flow -->
<!-- ⚠ BAD: <div tabindex="1">...</div> -->
```

10.2.7 contenteditable — Edit content

```
<!-- Allow content editing -->
<div contenteditable="true">
  This text can be edited by clicking
</div>

<!-- Read-only -->
<div contenteditable="false">
  This text cannot be edited
</div>
```

10.2.8 spellcheck — Spell checking

```
<!-- Enable spell checking -->
<textarea spellcheck="true"
  placeholder="Type here..."></textarea>

<!-- Disable spell checking -->
<input type="text" spellcheck="false">
```

10.2.9 draggable — Draggable

```
<!-- Make element draggable -->
<div draggable="true">
  I can be dragged
</div>


```

10.2.10 lang — Language

```
<!-- Specify language for accessibility and SEO -->
<html lang="en">
  <head>
    <title>My page</title>
  </head>
  <body>
    <p>Text in English</p>
    <p lang="es">Texto en español</p>
    <p lang="fr">Texte en français</p>
  </body>
</html>
```

10.2.11 dir — Text direction

```
<!-- Right-to-left text -->
<div dir="rtl">
  اليسار إلى اليمين من يذهب العربى النص
</div>

<!-- Left-to-right text (default) -->
<div dir="ltr">
  Text is read from left to right
</div>
```

10.3 Special attributes

10.3.1 aria-* — Accessibility (ARIA)

```
<!-- Define element role -->
<div role="button" aria-pressed="false">
  Simulated button
</div>

<!-- Label for screen readers -->
<button aria-label="Close menu">x</button>

<!-- Additional description -->
<input aria-describedby="pwd-hint">
<p id="pwd-hint">Minimum 8 characters</p>
```

10.3.2 rel — Relationship with external resources

```
<!-- Link to stylesheet -->
<link rel="stylesheet" href="styles.css">

<!-- Favicon -->
<link rel="icon" href="favicon.ico">

<!-- Preload resource -->
<link rel="preload" href="font.woff2" as="font">

<!-- Link to alternate version -->
<link rel="alternate" href="/es/" hreflang="es">
```

10.4 Advanced attributes checklist

- ✓ Use `data-*` for custom data
- ✓ Unique `id` per page
- ✓ `class` for CSS styles (not inline style)
- ✓ `title` for additional information
- ✓ `hidden` to hide elements
- ✓ `tabindex="0"` for interactive elements
- ✓ `lang` for multiple languages
- ✓ `aria-*` for accessibility
- ✓ `rel` for correct relationships

In the next chapter you'll learn about advanced **Meta Tags** for SEO, social media, and progressive web apps.

ADVANCED META TAGS (OPEN GRAPH, TWITTER CARDS, PWA)

Advanced meta tags control how your pages appear on **social media**, **search engines**, and **progressive web apps**.

11.1 Open Graph (OG)

Open Graph lets you control what **image**, **title**, and **description** appear when your page is shared on social media.

11.1.1 Open Graph meta tags

```
<head>
  <!-- Page title (for OG) -->
  <meta property="og:title" content="HTML Guide for Engineering">

  <!-- Description (for OG) -->
  <meta property="og:description" content="Learn HTML from scratch with practical
  examples">

  <!-- Image displayed (recommended 1200x630px) -->
  <meta property="og:image" content="https://example.com/cover.jpg">

  <!-- Page URL -->
  <meta property="og:url" content="https://example.com/html">

  <!-- Content type -->
  <meta property="og:type" content="website">

  <!-- Language (optional) -->
  <meta property="og:locale" content="en_US">
</head>
```

11.1.2 Complete example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>HTML Guide | USAL</title>

  <!-- Standard SEO meta -->
  <meta name="description" content="Interactive HTML tutorial for Computer
↪Engineering students">
  <meta name="keywords" content="HTML5, web development, tutorial">

  <!-- Open Graph for social media -->
  <meta property="og:title" content="HTML Guide for Computer Engineering">
  <meta property="og:description" content="Interactive HTML5 tutorial for USAL
↪students">
  <meta property="og:image" content="https://guide.example.com/og-image.jpg">
  <meta property="og:url" content="https://guide.example.com/html">
  <meta property="og:type" content="website">
  <meta property="og:site_name" content="HTML Guide USAL">

  <!-- Twitter Card -->
  <meta name="twitter:card" content="summary_large_image">
  <meta name="twitter:title" content="HTML Guide for Engineering">
  <meta name="twitter:description" content="Learn HTML from scratch">
  <meta name="twitter:image" content="https://guide.example.com/twitter-image.jpg">
  <meta name="twitter:site" content="@usal">
</head>
<body>
  <!-- Content... -->
</body>
</html>
```

11.2 Twitter Cards

Twitter Cards control how your content appears on Twitter/X when shared.

11.2.1 Card types

summary — Simple card

```
<meta name="twitter:card" content="summary">
<meta name="twitter:title" content="My article">
<meta name="twitter:description" content="Brief description">
<meta name="twitter:image" content="https://example.com/image.jpg">
```


summary_large_image — Large image

```
<meta name="twitter:card" content="summary_large_image">
<meta name="twitter:title" content="My article">
<meta name="twitter:description" content="Brief description">
<meta name="twitter:image" content="https://example.com/large-image.jpg">
```

player — For video/audio

```
<meta name="twitter:card" content="player">
<meta name="twitter:player" content="https://example.com/player">
<meta name="twitter:player:width" content="550">
<meta name="twitter:player:height" content="310">
```

11.2.2 Additional attributes

```
<!-- Twitter username -->
<meta name="twitter:creator" content="@your_user">
<meta name="twitter:site" content="@site_name">

<!-- Recommended image dimensions -->
<!-- summary: 200x200px minimum, 1MB max -->
<!-- summary_large_image: 506x506px minimum, 1MB max -->
```

11.3 PWA (Progressive Web App)

PWAs are web apps that work offline and can be installed on devices.

11.3.1 Manifest.json

```
{
  "name": "HTML Guide",
  "short_name": "HTML",
  "description": "Interactive HTML tutorial for engineering",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#1976d2",
  "icons": [
    {
      "src": "/icon-192x192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icon-512x512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

(continues on next page)

(continued from previous page)

```
]
}
```

11.3.2 Link manifest in HTML

```
<head>
  <link rel="manifest" href="/manifest.json">

  <!-- Status bar color on mobile -->
  <meta name="theme-color" content="#1976d2">

  <!-- Icon for Apple -->
  <link rel="apple-touch-icon" href="/icon-180x180.png">
</head>
```

11.3.3 Service Worker

```
<!-- Register Service Worker -->
<script>
  if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('/sw.js')
      .then(() => console.log('SW registered'))
      .catch(err => console.log('Error:', err));
  }
</script>
```

11.4 Mobile meta tags

```
<!-- Viewport (essential for responsive) -->
<meta name="viewport" content="width=device-width, initial-scale=1">

<!-- Allow installation on home screen -->
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
<meta name="apple-mobile-web-app-title" content="HTML Guide">

<!-- Address bar color in Android -->
<meta name="theme-color" content="#1976d2">

<!-- Disable zoom (use carefully) -->
<!-- <meta name="viewport" content="user-scalable=no"> -->
```

11.5 Complete SEO meta tags

```
<head>
  <!-- Basic meta -->
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">

  <!-- SEO -->
  <title>HTML Guide | USAL</title>
  <meta name="description" content="HTML tutorial for Computer Engineering">
  <meta name="keywords" content="HTML, web, tutorial, USAL">
  <meta name="author" content="Luis Augusto Silva">

  <!-- Open Graph -->
  <meta property="og:title" content="HTML Guide">
  <meta property="og:description" content="Interactive tutorial">
  <meta property="og:image" content="https://example.com/og.jpg">
  <meta property="og:url" content="https://example.com">
  <meta property="og:type" content="website">

  <!-- Twitter -->
  <meta name="twitter:card" content="summary_large_image">
  <meta name="twitter:title" content="HTML Guide">
  <meta name="twitter:description" content="Interactive tutorial">
  <meta name="twitter:image" content="https://example.com/twitter.jpg">

  <!-- PWA -->
  <link rel="manifest" href="/manifest.json">
  <meta name="theme-color" content="#1976d2">

  <!-- Favicon -->
  <link rel="icon" href="/favicon.ico">
  <link rel="apple-touch-icon" href="/apple-icon.png">

  <!-- Canonical (for duplicates) -->
  <link rel="canonical" href="https://example.com/html">
</head>
```

11.6 Validate meta tags

Useful tools:

- **Open Graph Debugger** — <https://developers.facebook.com/tools/debug/>
- **Twitter Card Validator** — <https://cards-dev.twitter.com/validator>
- **Google Search Console** — <https://search.google.com/search-console/>
- **Lighthouse** (in DevTools) — Audit PWA

11.7 Advanced meta tags checklist

- ✓ `og:title`, `og:description`, `og:image` for social media
- ✓ OG image: 1200x630px minimum
- ✓ Twitter Card type appropriate (`summary_large_image` recommended)
- ✓ `manifest.json` for PWA
- ✓ `theme-color` for mobile
- ✓ `viewport` for responsive
- ✓ `canonical` if duplicates exist
- ✓ `description` for SEO
- ✓ Validate with official tools

With these advanced meta tags, your page will have better **social media visibility**, **better SEO**, and can function as a **progressive web app**. Congratulations, you've completed the HTML guide!